

05506026 Digital Image Processing (Java)

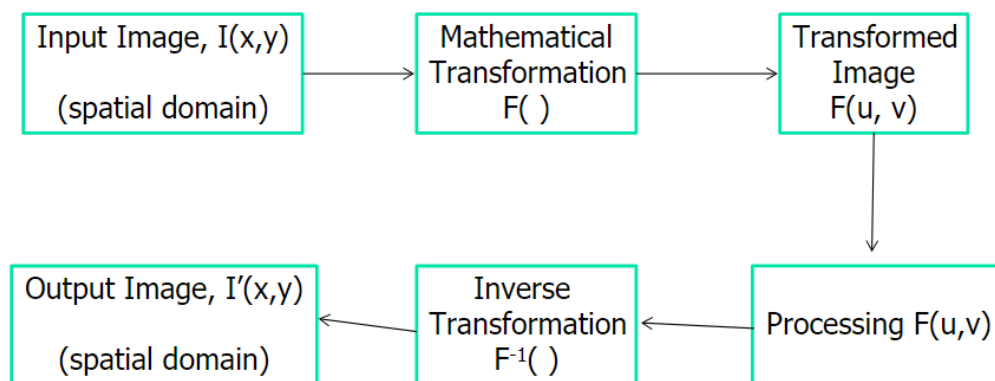
Lab 5

Objectives

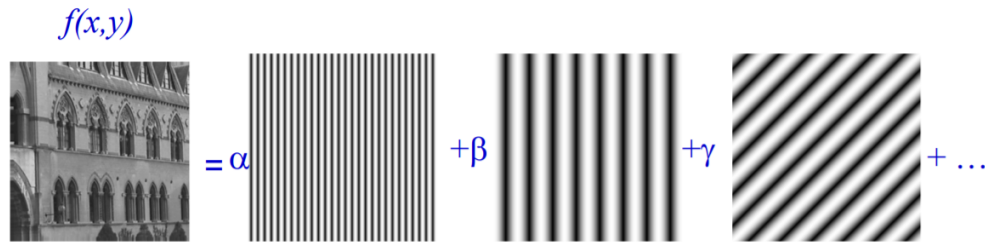
1. Understand how to convert images in spatial domain to frequency domain and convert back.
2. Try applying some basic low-pass and high-pass filter to the frequency domain.

Frequency Domain

Frequency Domain is another form of image, apart from spatial domain (matrix of pixels) we have been familiar with. Using Frequency Domain is easier way to deal with some tasks which is difficult to be executed using spatial domain. An image in spatial domain can be converted to frequency domain with Fourier Transforms. The image will be displayed in form of Fourier Series.



The Fourier Series of images are the summation series of several sine functions with different coefficients.



The Fourier Series of an image can be converted using Fourier Transforms. The Fourier Transforms for digital images must be discrete and 2 dimensional consist of two parts which are real numbers with cosine functions and imaginary numbers with sine functions. For a digital image with a size $M \times N$ is as follows.

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \left[\cos \left(2\pi \left(\frac{ux}{M} + \frac{vy}{N} \right) \right) - j \sin \left(2\pi \left(\frac{ux}{M} + \frac{vy}{N} \right) \right) \right]$$

The pseudo code of 1D Discrete Fourier Transform (DFT) is as follows. Note that we need to perform in 2 dimensions for images.

```

For all transform points ( u = [0, M-1] ) do
    F(u).real = 0
    F(u).im = 0
    For all discrete data points ( x = [0, M-1] ) do
        F(u).real += F(x) cos( $\frac{2\pi ux}{M}$ )
        F(u).im -= F(x) sin( $\frac{2\pi ux}{M}$ )
    End For
    F(u).real = sum
    F(u).im = sum
End For

```

On the other hand, we can inverse the Fourier Series back to spatial image by using inverse function of Fourier Transforms.

$$f(x, y) = \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} F(u, v) e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

Which can be written as pseudo code (1D) as follows.

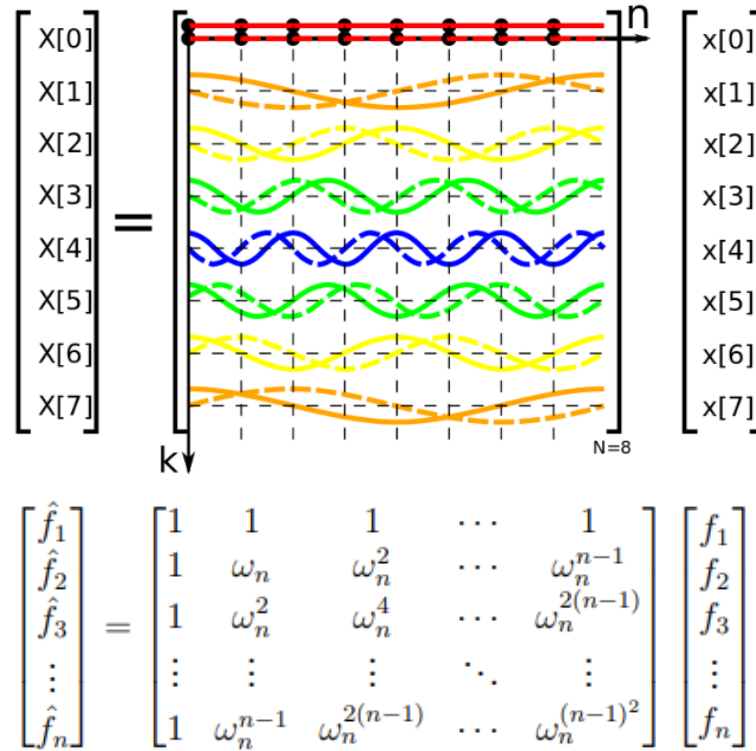
```

For all transform points ( x = [0, M-1] ) do
    F(u).real = 0
    F(u).im = 0
    For all discrete data points ( u = [0, M-1] ) do
        F(u).real += F(x) cos( $\frac{2\pi ux}{M}$ )
        F(u).im += F(x) sin( $\frac{2\pi ux}{M}$ )
    End For
    F(u).real =  $\frac{sum}{M}$ 
    F(u).im =  $\frac{sum}{M}$ 
End For

```

In the real implementation, this DFT is too slow (complexity $O(n^2)$ on each dimension). What we will be using here is an improved version of DFT called Fast Fourier Transform (FFT) which is one of the most powerful algorithms of the last century. There are a lot of FFT algorithms in practical use, but the most common algorithm is called Cooley-Tukey FFT algorithm.

The idea of the Cooley-Tukey FFT algorithm comes from the basis of DFT computation is a multiplication of matrices of different coefficient signals.



Since the coefficient matrix W above is symmetrical, if our matrix is 1024×1024 , we can rearrange the multiplication below.

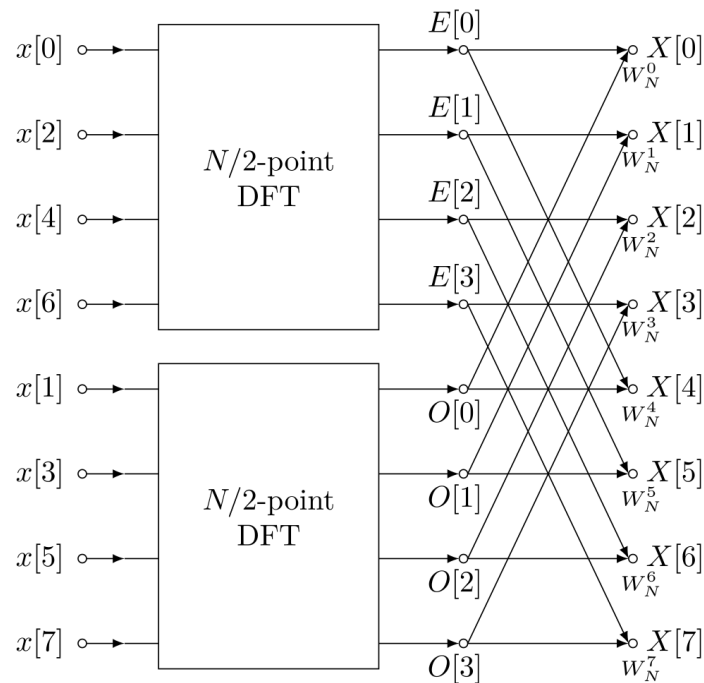
$$\hat{f} = F_{1024} f = \begin{bmatrix} I_{512} & -D_{512} \\ I_{512} & D_{512} \end{bmatrix} \begin{bmatrix} F_{512} & 0 \\ 0 & F_{512} \end{bmatrix} \begin{bmatrix} f_{even} \\ f_{odd} \end{bmatrix}$$

Where I_{512} is the 512×512 identity matrix, f_{even} are the even indices of function matrix f , f_{odd} are the odd indices of function matrix f , and D_{512} is the diagonal matrix given by: -

$$D_{512} = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & \omega & 0 & \cdots & 0 \\ 0 & 0 & \omega^2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \omega^{511} \end{bmatrix}$$

From here, we can go further again and again recursively until 1×1 matrix ($F_{1024} \rightarrow F_{512} \rightarrow F_{256} \rightarrow \dots \rightarrow F_4 \rightarrow F_2 \rightarrow F_1$), which means the computational time will be significantly decreased (identity

matrix multiplied by bunch-of-zeroes matrix). The Cooley-Tukey FFT algorithm is commonly visualised as a butterfly pattern below.



This reduces the computational complexity from $O(n^2)$ to $O(n \log n)$. However, this works only when the matrix is the power of 2. But we can pad our matrix to the next power of 2 with 0s. The FFT can be written as pseudo code (1D) below.

```

function FFT(input_sequence):
    N = length(input_sequence)
    if N == 1:
        return input_sequence

    even_values = FFT(input_sequence[even indices])
    odd_values = FFT(input_sequence[odd indices])

    output_sequence = array of length N

    for k from 0 to N/2 - 1:
        theta = -2 *  $\pi$  * k / N
        cos_theta = cos(theta)
        sin_theta = sin(theta)

        t_real = cos_theta * odd_values[k][0] - sin_theta * odd_values[k][1]
        t_imag = cos_theta * odd_values[k][1] + sin_theta * odd_values[k][0]

        output_sequence[k] = [even_values[k][0] + t_real, even_values[k][1] + t_imag]
        output_sequence[k + N/2] = [even_values[k][0] - t_real, even_values[k][1] - t_imag]

    return output_sequence

```

We will start implementing the code here. In Java, we need something to store the complex numbers and calculate them. We will start a new file for the frequency domain processing as below.

FrequencyDomainManager.java

```
class FrequencyDomainManager
{
}

class Complex
{
    public double re;
    public double im;

    public Complex()
    {
        this(0, 0);
    }

    public Complex(double re, double im)
    {
        this.re = re;
        this.im = im;
    }

    public Complex add(Complex b)
    {
        return new Complex(this.re + b.re, this.im + b.im);
    }

    public Complex sub(Complex b)
    {
        return new Complex(this.re - b.re, this.im - b.im);
    }

    public Complex mult(Complex b)
    {
        return new Complex((this.re * b.re) - (this.im * b.im),
            (this.re * b.im) + (this.im * b.re));
    }

    public Complex conjugate()
    {
        return new Complex(re, -im);
    }

    public Complex scale(double b)
    {
        return new Complex(re * b, im * b);
    }

    public double length()
    {
        return Math.sqrt((re * re) + (im * im));
    }
}
```

The transformed frequency domain image will be stored separately with the different width and height of the original image. The width and height of the transformed image are the next power of 2 of the original values. For example, the width of transformed image with 200 pixels will be 256 pixels, the width of transformed image with 500 pixels will be 512 pixels, etc.

These will be placed in the constructor method as follows.

```
FrequencyDomainManager.java
class FrequencyDomainManager
{
    Complex[][] img;
    int width, height, type;
    int imgWidth, imgHeight;
    private Complex[][] original;
    ImageManager im;

    public FrequencyDomainManager(ImageManager im)
    {
        this.im = im;

        imgWidth = im.width;
        imgHeight = im.height;
        width = nextPowerOf2(imgWidth);
        height = nextPowerOf2(imgHeight);
        type = im.getImage().getType();

        img = new Complex[height][width];
    }

    public static int nextPowerOf2(final int a)
    {
        int b = 1;
        while (b < a)
        {
            b = b << 1;
        }
        return b;
    }
}
```

We will implement the 1D FFT algorithm from an array of complex number according to the pseudo code above as follows.

FrequencyDomainManager.java

```

class FrequencyDomainManager
{
    ...
    public static Complex[] fft(Complex[] x)
    {
        int n = nextPowerOf2(x.length);

        //padding
        Complex[] fftArray = new Complex[n];
        for (int i = 0; i < n; i++)
        {
            fftArray[i] = new Complex();
            if(i < x.length)
            {
                fftArray[i].re = x[i].re;
                fftArray[i].im = x[i].im;
            }
        }

        // base case
        if (n == 1) return new Complex[] { fftArray[0] };

        // compute FFT of even terms
        Complex[] even = new Complex[n/2];
        for (int k = 0; k < n/2; k++) {
            even[k] = fftArray[2*k];
        }
        Complex[] evenFFT = fft(even);

        // compute FFT of odd terms
        Complex[] odd = even; // reuse the array (to avoid n log n space)
        for (int k = 0; k < n/2; k++) {
            odd[k] = fftArray[2*k + 1];
        }
        Complex[] oddFFT = fft(odd);

        // combine
        Complex[] y = new Complex[n];
        for (int k = 0; k < n/2; k++) {
            double kth = -2 * k * Math.PI / n;
            Complex wk = new Complex(Math.cos(kth), Math.sin(kth));
            y[k] = evenFFT[k].add(wk.mult(oddFFT[k]));
            y[k + n/2] = evenFFT[k].sub(wk.mult(oddFFT[k]));
        }
        return y;
    }
    ...
}

```

For Fourier Transform on digital images, we need 2D FFT which convert arrays horizontally then vertically, or vice versa. Therefore, the 2D FFT will be implemented as follows.

```
FrequencyDomainManager.java
class FrequencyDomainManager
{
    ...
    private void fft2d()
    {
        Complex[] temp = new Complex[width];
        //horizontal first
        for (int y = 0; y < height; y++)
        {
            for (int u = 0; u < width; u++)
            {
                temp[u] = new Complex();
                temp[u].re = img[y][u].re;
                temp[u].im = img[y][u].im;
            }

            temp = fft(temp);

            for (int u = 0; u < width; u++)
            {
                img[y][u].re = temp[u].re;
                img[y][u].im = temp[u].im;
            }
        }

        //then vertical
        temp = new Complex[height];
        for (int x = 0; x < width; x++)
        {
            for (int v = 0; v < height; v++)
            {
                temp[v] = new Complex();
                temp[v].re = img[v][x].re;
                temp[v].im = img[v][x].im;
            }

            temp = fft(temp);

            for (int v = 0; v < height; v++)
            {
                img[v][x].re = temp[v].re;
                img[v][x].im = temp[v].im;
            }
        }
    }
}
```

Therefore, we can transform the image into frequency domain and store it as follows.

```

FrequencyDomainManager.java
class FrequencyDomainManager
{
    ...
    public FrequencyDomainManager(ImageManager im)
    {
        ...
        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                int color = im.getImage().getRGB(x, y);
                int gray = color & 0xff;

                img[y][x] = new Complex(gray, 0);
            }
        }

        fft2d();

        //store original
        original = new Complex[height][width];
        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                original[y][x] = new Complex(img[y][x].re, img[y][x].im);
            }
        }
    }
    ...
}

```

With this, we can convert from our image in ImageManager into a frequency domain as follows.

ImageManager.java

```

class ImageManager
{
    ...
    public FrequencyDomainManager getFrequencyDomain()
    {
        convertToGrayscale();
        FrequencyDomainManager fft = new FrequencyDomainManager(this);
        restoreToOriginal();

        return fft;
    }
    ...
}

```

Writing images in Spectrum and Phase

The frequency domain image is stored in array of complex numbers consisting of real and imaginary numbers. To visualise this, we will use spectrum and phase which can be calculated as follows.

$$\text{Magnitude} = \sqrt{\text{Re}^2(u) + \text{Im}^2(u)}$$

$$\text{Phase} = \theta(u) = \tan^{-1} \left[\frac{\text{Im}(u)}{\text{Re}(u)} \right]$$

We will implement 2 methods to write spectrum (in log 10 scale) and phase images as follows.

FrequencyDomainManager.java

```

class FrequencyDomainManager
{
    ...
    public boolean writeSpectrumLogScaled(String fileName)
    {
        try
        {
            BufferedImage tempimg = new BufferedImage(width, height, type);

            double max = Double.MIN_VALUE, min = Double.MAX_VALUE;

```

```

        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                double spectrum = img[y][x].length();

                if (spectrum > max) max = spectrum;
                if (spectrum < min) min = spectrum;
            }
        }

        min = min < 1.0f? 0f: Math.log10(min);
        max = max < 1.0f? 0f: Math.log10(max);

        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                double spectrum = img[y][x].length();

                spectrum = spectrum < 1.0f? 0f: Math.log10(spectrum);
                spectrum = ((spectrum-min)*255 / (max-min));
                int color = ((int)spectrum << 16) | ((int)spectrum << 8) | (int)spectrum;

                tempimg.setRGB(x, y, color);
            }
        }

        BufferedImage img = new BufferedImage(width, height, type);

        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                img.setRGB(x, y, tempimg.getRGB(x, y));
            }
        }

        ImageIO.write(img, "bmp", new File (fileName));
        System.out.println("Image "+fileName+" has been written!");

        return true;
    }
    catch (IOException e)
    {
        System.out.println(e);

        return false;
    }
}
...
}

```

FrequencyDomainManager.java

```

class FrequencyDomainManager
{
    ...
    public boolean writePhase(String fileName)
    {
        try
        {
            BufferedImage imgOut = new BufferedImage(width, height, type);

            double max = Double.MIN_VALUE, min = Double.MAX_VALUE;

            for (int y = 0; y < height; y++)
            {
                for (int x = 0; x < width; x++)
                {
                    double phase = (Math.atan2(img[y][x].im, img[y][x].re));

                    if (phase > max) max = phase;
                    if (phase < min) min = phase;
                }
            }

            for (int y = 0; y < height; y++)
            {
                for (int x = 0; x < width; x++)
                {
                    double phase = (Math.atan2(img[y][x].im, img[y][x].re));

                    phase = ((phase-min)*255 / (max-min));

                    int color = ((int)phase << 16) | ((int)phase << 8) | (int)phase;

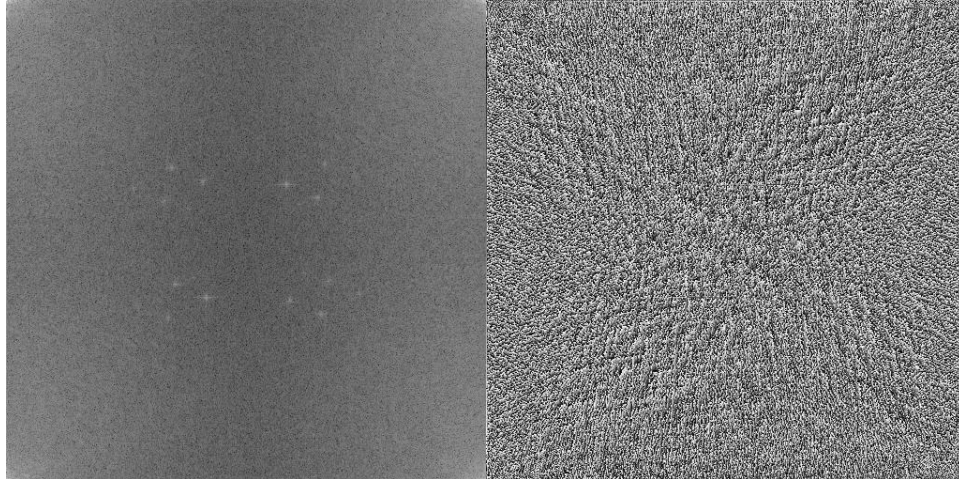
                    imgOut.setRGB(x, y, color);
                }
            }
            ImageIO.write(imgOut, "bmp", new File (fileName));
            System.out.println("Image "+fileName+" has been written!");

            return true;
        }
        catch (IOException e)
        {
            System.out.println(e);

            return false;
        }
    }
    ...
}

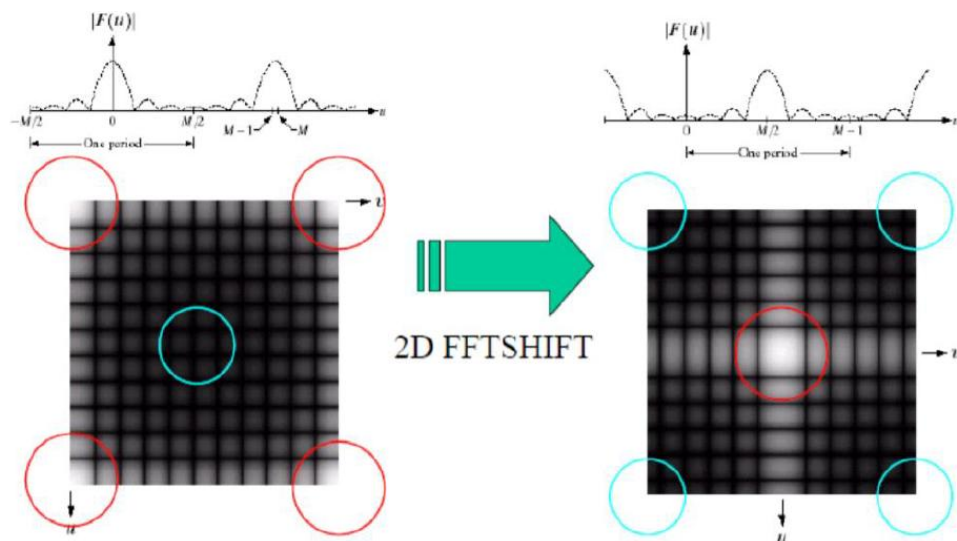
```

Below are the spectrum and phase of frequency domain of Mandril.bmp.



Shifting the coefficient

Normally, the frequency domain will store the low frequency in the edge and high frequency in the middle. But images normally have more low frequency (background colours) than high frequency (edges). Therefore, it is better to shift the low frequency to the middle for better observation.



This can be implemented as follows.

FrequencyDomainManager.java

```

class FrequencyDomainManager
{
    ...
    public FrequencyDomainManager(ImageManager im)
    {
        ...
        fft2d();
        shifting();
        ...
    }
    ...
    private void shifting()
    {
        int halfWidth = width / 2;
        int halfHeight = height / 2;

        for (int y = 0; y < halfHeight; y++)
        {
            for (int x = 0; x < width; x++)
            {
                Complex temp = new Complex(img[y][x].re, img[y][x].im);

                img[y][x].re = img[y + halfHeight][x].re;
                img[y][x].im = img[y + halfHeight][x].im;

                img[y + halfHeight][x].re = temp.re;
                img[y + halfHeight][x].im = temp.im;
            }
        }

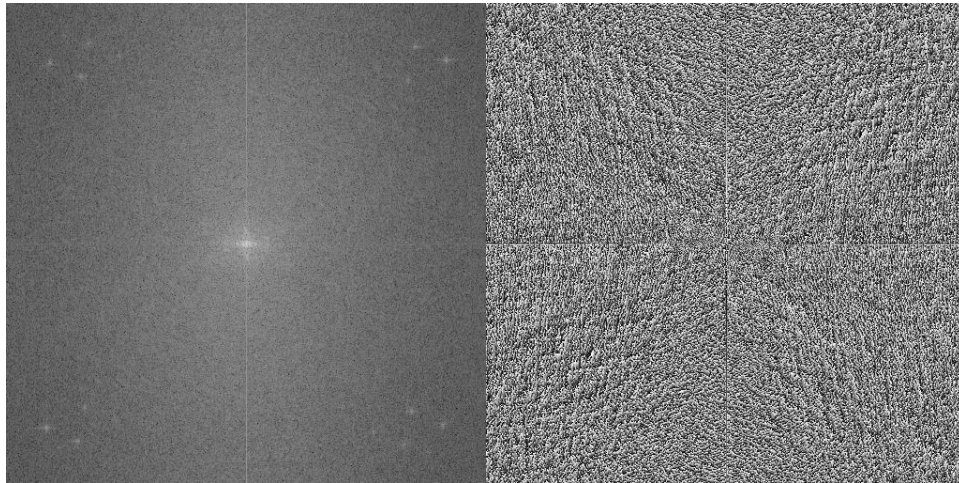
        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < halfWidth; x++)
            {
                Complex temp = new Complex(img[y][x].re, img[y][x].im);

                img[y][x].re = img[y][x + halfWidth].re;
                img[y][x].im = img[y][x + halfWidth].im;

                img[y][x + halfWidth].re = temp.re;
                img[y][x + halfWidth].im = temp.im;
            }
        }
    }
    ...
}

```


Below are the shifted spectrum and phase of frequency domain of Mandril.bmp.



Inverse the frequency domain

The stored frequency numbers can be inverted back to normal colour values using inverse Fourier Transform as follows.

```
FrequencyDomainManager.java
class FrequencyDomainManager
{
    ...
    public FrequencyDomainManager(ImageManager im)
    {
        ...
        fft2d(false);
        ...
    }
    ...
    private void fft2d(boolean invert)
    {
        ...
        //horizontal first
        for (int y = 0; y < height; y++)
        {
            ...
            if (!invert) temp = fft(temp);
            else temp = ifft(temp);
            ...
        }
        //then vertical
        temp = new Complex[height];
        for (int x = 0; x < width; x++)
        {
            ...
            if (!invert) temp = fft(temp);
```

```

        else temp = ifft(temp);
        ...
    }
}
...

public void getInverse()
{
    shifting();
    fft2d(true);

    for (int y = 0; y < imgHeight; y++)
    {
        for (int x = 0; x < imgWidth; x++)
        {
            int gray = (int) img[y][x].re;
            gray = gray > 255? 255: gray;
            gray = gray < 0? 0: gray;

            int color = (gray << 16) | (gray << 8) | gray;

            im.getImage().setRGB(x, y, color);
        }
    }
}

// compute the inverse FFT of x[]
public static Complex[] ifft(Complex[] x)
{
    int n = x.length;
    Complex[] y = new Complex[n];

    // take conjugate
    for (int i = 0; i < n; i++) {
        y[i] = x[i].conjugate();
    }

    // compute forward FFT
    y = fft(y);

    // take conjugate again
    for (int i = 0; i < n; i++) {
        y[i] = y[i].conjugate();
    }

    // divide by n
    for (int i = 0; i < n; i++) {
        y[i] = y[i].scale(1.0 / n);
    }

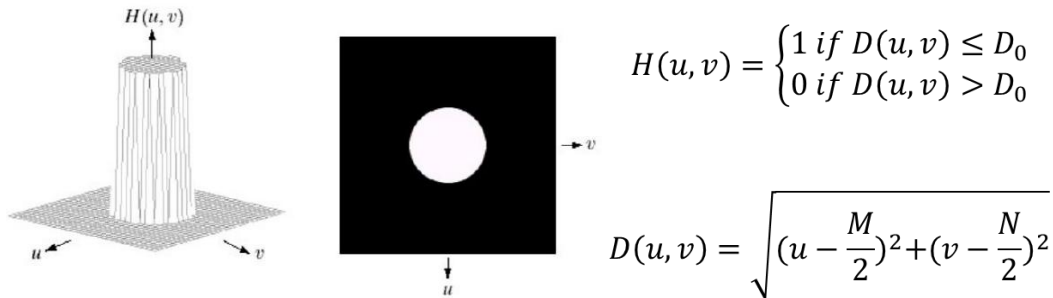
    return y;
}

...
}

```

Ideal Lowpass Filter

Lowpass filter is the algorithm to smooth the image. The concept is to remove the high frequency signals of the image. The simplest method is called ideal lowpass filter which remove (make the values to 0s) the frequency higher than a threshold value. In the frequency domain case, the high frequency is at the edges of the array.



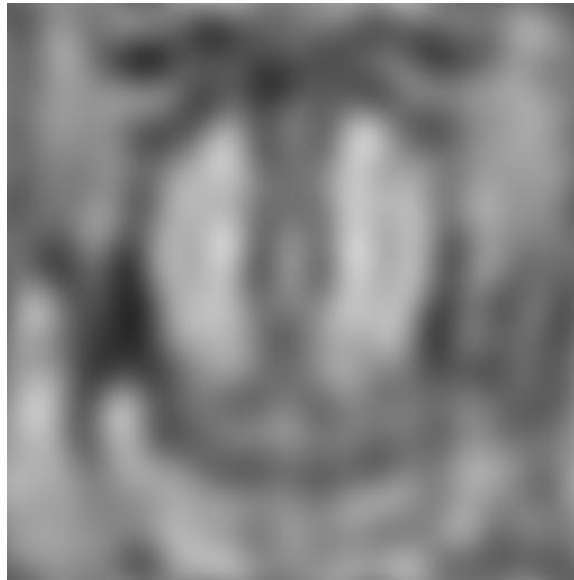
This can be implemented as follows.

```
FrequencyDomainManager.java
class FrequencyDomainManager
{
    ...
    public void ILPF(double radius)
    {
        if (radius <= 0 || radius > Math.min(width/2, height/2))
        {
            System.out.println("INVALID Radius!");
            return;
        }

        int centerX = width/2;
        int centerY = height/2;

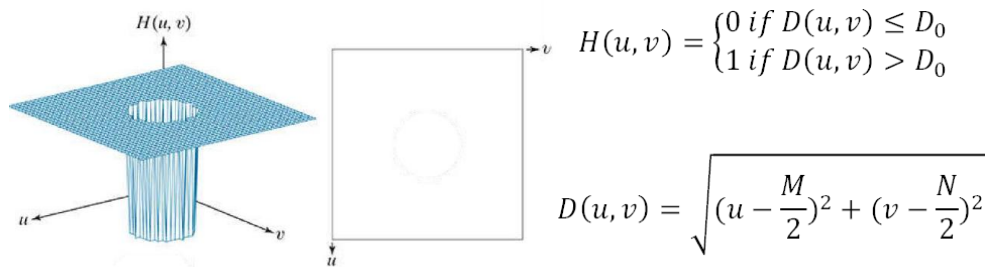
        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                if ((x - centerX) * (x - centerX) + (y - centerY) * (y - centerY) > radius *
radius)
                {
                    img[y][x].re = 0;
                    img[y][x].im = 0;
                }
            }
        }
        ...
    }
}
```

Below is ILPF with radius 10 of Mandril.bmp.



Ideal Highpass Filter

Highpass filter is the algorithm to sharpen the image. The concept is to remove the low frequency signals of the image. The simplest method is called ideal lowpass filter which remove (make the values to 0s) the frequency lower than a threshold value. In the frequency domain case, the low frequency is at the middle of the array.



This can be implemented as follows.

FrequencyDomainManager.java

```

class FrequencyDomainManager
{
    ...
    public void IHPF(double radius)
    {
        if (radius <= 0 || radius > Math.min(width/2, height/2))
        {
            System.out.println("INVALID Radius!");
            return;
        }

        int centerX = width/2;
        int centerY = height/2;

        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                if ((x - centerX) * (x - centerX) + (y - centerY) * (y - centerY) <= radius *
radius)
                {
                    img[y][x].re = 0;
                    img[y][x].im = 0;
                }
            }
        }
        ...
    }
}

```

Below is IHPF with radius 10 of Mandril.bmp.



Quest 008

Implement the method of **ILPF** and perform the radius of 3, 5, 10, and 20

The player who finishes this task will be awarded **30 EXP** and **2 Moves**.

Deadline: 7 September 2025